

While Loops in Python

Unknown Repetition, Validation and Menus

Programming Fundamentals Team

SETU

2026-06-03

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

1.1 By the end of this lecture

You should be able to:

- Explain when a `while` loop is useful
- Write a simple `while` loop in Python
- Use counters and accumulators
- Write validation loops
- Write sentinel-controlled loops
- Create a simple menu loop
- Identify and avoid infinite loops

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

2.1 You already know selection

```
age = int(input("Enter age: "))

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Selection makes a decision once.

2.2 What if we need to keep deciding?

Example:

- Keep asking for a password until it is correct.
- Keep asking for a mark until it is between 0 and 100.
- Keep displaying a menu until the user chooses Exit.

A loop lets us repeat a block of code.

2.3 Selection versus while loop

```
if condition:
    print("Runs once if condition is true")

while condition:
    print("Runs again and again while condition is true")
```

The condition is checked before each repetition.

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

3.1 Syntax

```
while condition:  
    statement_1  
    statement_2
```

Important:

- The colon : starts the loop body.
- Indentation shows which statements belong inside the loop.

3.2 First example

```
count = 1

while count <= 5:
    print("Hello")
    count += 1
```

This prints Hello five times.

3.3 Anatomy of the loop

```
count = 1           # setup

while count <= 5:   # test
    print(count)    # repeated action
    count += 1      # update
```

Setup, test, action, update.

3.4 Trace table

```
count = 1
```

```
while count <= 3:  
    print(count)  
    count += 1
```

count	condition	output
1	True	1
2	True	2
3	True	3
4	False	Stop

3.5 Activity: predict the output

```
x = 3
```

```
while x <= 9:  
    print(x)  
    x += 2
```

Questions:

- What is the first value printed?
- What is the last value printed?
- Why does the loop stop?

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

4.1 Count from 1 to 10

```
i = 1

while i <= 10:
    print(i)
    i += 1
```


4.2 Count backwards

```
i = 10

while i >= 1:
    print(i)
    i -= 1
```

4.3 Count by twos

```
i = 0

while i <= 20:
    print(i)
    i += 2
```

4.4 Quick challenge

Write a `while` loop that prints:

5
10
15
20
25
30

4.5 Quick challenge Solution

```
x = 5
```

```
while x <= 30:  
    print(x)  
    x += 5
```

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

5.1 Counter pattern

A counter keeps track of how many times something happens.

```
count = 0  
count += 1
```

Examples:

- number of valid marks entered
- number of failed login attempts
- number of products sold

5.2 Accumulator pattern

An accumulator builds up a total.

```
total = 0  
total += value
```

Examples:

- total marks
- total cost
- total distance

5.3 Sum numbers 1 to 5

```
total = 0
num = 1

while num <= 5:
    total += num
    num += 1

print(total)
```

Output:

15

5.4 Trace the accumulator

num	total before	total after
1	0	1
2	1	3
3	3	6
4	6	10
5	10	15

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

6.1 Why validation loops?

User input cannot always be trusted.

Examples:

- A mark must be between 0 and 100.
- A menu choice must be one of the options.
- A password must match.

A validation loop repeats until the input is acceptable.

6.2 Validating a mark

```
mark = int(input("Enter mark: "))

while mark < 0 or mark > 100:
    print("Invalid mark")
    mark = int(input("Enter mark: "))

print("Accepted")
```

This uses `or`, which students have already met.

6.3 Validating a positive number

```
number = int(input("Enter a positive number: "))

while number <= 0:
    print("That is not positive")
    number = int(input("Try again: "))

print("Thank you")
```

6.4 Password example

```
password = input("Password: ")

while password != "python":
    password = input("Incorrect. Try again: ")

print("Access granted")
```

6.5 Activity: fix the validation

This code accepts invalid marks. Why?

```
mark = int(input("Mark: "))

while mark < 0 and mark > 100:
    print("Invalid")
    mark = int(input("Mark: "))
```

Hint: think about and versus or.

6.6 Activity: Solution

This code accepts invalid marks. Why?

```
mark = int(input("Mark: "))

while mark < 0 or mark > 100:
    print("Invalid")
    mark = int(input("Mark: "))
```


Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

7.1 What is a sentinel?

A sentinel is a special value that means stop.

Examples:

- -1 to stop entering numbers
- quit to stop entering commands
- 0 to stop entering product codes

7.2 Print numbers until -1

```
number = int(input("Number (-1 to stop): "))

while number != -1:
    print(number)
    number = int(input("Number (-1 to stop): "))
```

7.3 Total numbers until -1

```
total = 0
number = int(input("Number (-1 to stop): "))

while number != -1:
    total += number
    number = int(input("Number (-1 to stop): "))

print("Total:", total)
```

7.4 Count numbers until -1

```
count = 0
number = int(input("Number (-1 to stop): "))

while number != -1:
    count += 1
    number = int(input("Number (-1 to stop): "))

print("You entered", count, "numbers")
```

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

8.1 Why menu loops?

Many programs keep running until the user chooses to exit.

Examples:

- ATM menu
- shop system
- student record system
- game menu

8.2 Basic menu loop

```
choice = ""

while choice != "4":
    print("1. Add")
    print("2. Search")
    print("3. Report")
    print("4. Exit")

    choice = input("Choice: ")
```


8.3 Menu with selection

```
choice = ""

while choice != "4":
    print("1. Add")
    print("2. Search")
    print("3. Report")
    print("4. Exit")

    choice = input("Choice: ")

    if choice == "1":
        print("Add selected")
    elif choice == "2":
        print("Search selected")
    elif choice == "3":
```

8.3 Menu with selection

```
    print("Report selected")  
elif choice != "4":  
    print("Invalid choice")
```

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

9.1 Common cause

```
i = 1

while i <= 5:
    print(i)
```

The update is missing.

i never changes, so the condition never becomes false.

9.2 Intentional infinite loop

```
while True:
    command = input("Enter command: ")

    if command == "quit":
        break

    print("You entered", command)
```

It is better to write the loop condition so that the loop naturally stops, rather than using break

9.3 Better without break

```
command = input("Enter command: ")

while command != "quit":
    print("You entered", command)
    command = input("Enter command: ")

print("Quitting")
```

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

10.1 Mini-program: average valid marks

```
total = 0
count = 0

while count < 5:
    mark = int(input("Enter mark: "))

    while mark < 0 or mark > 100:
        mark = int(input("Invalid. Try again: "))

    total += mark
    count += 1

average = total / count
print("Average:", average)
```


10.2 What does this combine?

- Input and output
- Selection-style conditions
- or
- Nested `while` loop
- Counter
- Accumulator
- Average calculation

Outline

1. Learning outcomes

2. From selection to iteration

3. Basic while loops

4. Counting loops

5. Accumulators and counters

6. Validation loops

7. Sentinel loops

8. Menu loops

9. Infinite loops

10. Bringing it together

11. Summary

11.1 Today we learned

- while loops repeat while a condition is true.
- Counters count events.
- Accumulators build totals.
- Validation loops keep asking until input is valid.
- Sentinel loops stop when a special value is entered.
- Menu loops keep programs running.
- Infinite loops often happen when the update is missing.

11.2 Practice exercises

1. Print your name 10 times.
2. Count from 1 to 20.
3. Count backwards from 20 to 1.
4. Print even numbers from 2 to 50.
5. Calculate the sum of numbers from 1 to 100.
6. Ask repeatedly for a positive number.
7. Ask for marks until -1 is entered, then print the average.
8. Create a menu that repeats until the user chooses Exit.